

Instituto Politécnico de Viana do Castelo

COMMOV

Sistema de Gestão de Projetos

Relatório Final de Projeto

Bruno Rosário - 26207 ; Gonçalo Marques - 31415 ; Rafael Monteiro - 27692
/ 3A - Grupo G

Ano letivo 2025/2026

1 de maio de 2026

Conteúdo

1	Introdução	3
2	Atualização dos Requisitos	4
2.1	Requisitos Funcionais	4
2.2	Requisitos Não Funcionais	7
2.3	Requisitos adicionais identificados durante o desenvolvimento	8
3	Modelo Final da Base de Dados	9
3.1	Descrição das tabelas	10
3.2	Relações entre as tabelas	12
3.3	Alterações face ao modelo intermédio	12
4	Arquitetura Final da Solução	14
4.1	Visão geral	14
4.2	Backend	14
4.3	Frontend Android	15
4.4	Autenticação e autorização	15
5	Tecnologias e Bibliotecas Utilizadas	16
6	Principais Funcionalidades Implementadas	16
6.1	Onboarding e autenticação	16
6.2	Dashboard	16
6.3	Gestão de projetos	17
6.4	Gestão de tarefas	17
6.5	Administração	18
6.6	Estatísticas e exportação	18
6.7	Definições	18
7	Funcionalidades Extra Desenvolvidas	18

8	Justificação das Alterações face à Entrega Intermédia	19
9	Capturas de Ecrã da Aplicação	20
10	Ligações de Apoio	23
10.1	Mockups e protótipos (Figma)	23
10.2	Repositório Git	23
10.3	Plataforma de gestão de projecto (Trello)	23
11	Testes e Validações Efectuadas	24
11.1	Testes automatizados no backend	24
11.2	Testes no frontend	24
11.3	Validação manual	24
11.4	Resultados gerais	24
12	Instruções de Instalação, Execução e Utilização	25
12.1	Pré-requisitos	25
12.2	Arranque do backend	25
12.3	Configuração e execução da app Android	25
12.4	Utilização básica	26
13	Conclusão	26

1 Introdução

A gestão de projetos e tarefas é uma necessidade cada vez mais comum em contextos académicos, profissionais e organizacionais. Quando existem equipas com diferentes responsabilidades, prazos a cumprir e várias tarefas a decorrer em simultâneo, torna-se importante ter uma ferramenta que ajude a centralizar informação, organizar o trabalho e acompanhar o progresso de cada atividade. Além disso, tendo em conta a utilização cada vez mais frequente de dispositivos móveis, faz sentido que este tipo de solução possa ser usada de forma simples e imediata através de uma aplicação móvel.

No âmbito da unidade curricular de *Computação Móvel*, do 3.º ano do curso de Engenharia Informática do Instituto Politécnico de Viana do Castelo, foi desenvolvido o projeto **COMMOV** (*ComMov*), uma aplicação orientada para a gestão de projetos, tarefas e utilizadores. O sistema foi pensado para ser utilizado por diferentes perfis, nomeadamente *administrador*, *gestor de projeto* e *utilizador*, permitindo que cada um tenha acesso às funcionalidades adequadas ao seu papel dentro da aplicação.

O principal objetivo do projeto é disponibilizar uma plataforma que permita criar e acompanhar projetos, gerir tarefas, atribuir utilizadores, registar o progresso do trabalho e consultar informação relevante de forma organizada. A interação com o utilizador é feita através da aplicação móvel Android **Velotask**, desenvolvida em Kotlin com Jetpack Compose e Material Design 3, que comunica com o servidor através de pedidos HTTP em formato JSON.

Este relatório corresponde à versão final do projeto e dá continuidade ao relatório intermédio entregue anteriormente. Na fase intermédia, o desenvolvimento encontrava-se sobretudo focado na base técnica da aplicação, principalmente no *backend*. Nessa altura, já existia uma estrutura inicial em Go, ligação a uma base de dados PostgreSQL, migrações, repositórios de acesso aos dados e alguns *endpoints* HTTP funcionais. No entanto, a aplicação móvel ainda não se encontrava totalmente integrada com o servidor e várias funcionalidades estavam ainda numa fase inicial ou por implementar.

Na fase final, o projeto evoluiu para uma solução mais completa, composta por três partes principais: uma API REST desenvolvida em Go, uma base de dados relacional PostgreSQL e a aplicação Android Velotask. A API foi organizada de forma estruturada, com separação entre rotas, serviços, repositórios e entidades, facilitando a manutenção e evolução do código. A base de dados passou a representar de forma mais completa os dados necessários ao funcionamento da aplicação, sendo gerida com GORM e migrações Goose. A aplicação móvel, por sua vez, passou a comunicar com o servidor e a disponibilizar ao utilizador as principais funcionalidades do sistema.

A solução desenvolvida segue uma arquitetura cliente-servidor, adequada a uma apli-

cação móvel com um *backend* centralizado. O acesso ao sistema é protegido através de autenticação por *token* do tipo *Bearer*, sendo as *passwords* armazenadas de forma segura através de *hash* bcrypt. Os diferentes perfis de utilizador permitem controlar o acesso às funcionalidades, garantindo que cada utilizador apenas consegue realizar as ações correspondentes ao seu papel. O servidor foi também preparado para ser executado com Docker Compose, o que facilita a instalação, a configuração do ambiente e o trabalho em equipa.

Ao longo da fase final foram também adicionadas funcionalidades que não estavam totalmente previstas ou concluídas na entrega intermédia, como o histórico de registos de tempo por tarefa, a possibilidade de associar vários utilizadores à mesma tarefa, a exportação de relatórios estatísticos em PDF, a gestão de fotografias de perfil e melhorias gerais na interface da aplicação móvel. Estas funcionalidades permitiram tornar o projeto mais completo e aproximá-lo de uma solução que poderia ser utilizada num contexto real.

O desenvolvimento foi realizado por um grupo de três elementos: Bruno Rosário, Gonçalo Marques e Rafael Monteiro. Para organizar o trabalho foram utilizadas ferramentas como o Trello, para planeamento e distribuição de tarefas, o Git/GitHub, para controlo de versões, e a ferramenta Bruno, para testar e validar os *endpoints* da API.

Este documento apresenta a evolução do projeto desde a fase intermédia até à versão final. Ao longo do relatório são descritos os objetivos da aplicação, o âmbito do sistema, as tecnologias utilizadas, os requisitos funcionais e não funcionais, o modelo final da base de dados, a arquitetura da solução, as principais funcionalidades implementadas, as diferenças face à entrega anterior, as instruções de instalação e utilização, os testes realizados e as dificuldades encontradas durante o desenvolvimento. Por fim, são apresentadas as conclusões do trabalho desenvolvido e algumas possibilidades de melhoria futura, como a implementação de uma vista Kanban, pesquisa global e registo autónomo de utilizadores.

2 Atualização dos Requisitos

2.1 Requisitos Funcionais

A Tabela 1 apresenta o estado final dos requisitos funcionais definidos na fase intermédia, indicando o grau de implementação e eventuais notas de atualização.

Tabela 1: Requisitos funcionais – estado final

ID	Descrição	Estado	Notas
RF01	Apresentar <i>intro sliders</i> ao utilizador.	Parcial	Onboarding com 4 ecrãs; persistência local com Room.
RF02	Permitir a criação de conta.	Parcial	Contas criadas pelo administrador; não existe registo público/autónomo.
RF03	Permitir o início de sessão.	Parcial	Autenticação por email/password com token Bearer.
RF04	Gestão do perfil (nome, username, email, fotografia, password).	Parcial	Fotografia e sessão implementadas; edição completa do perfil reservada ao administrador.
RF05	Suportar perfis Administrador, Gestor de Projeto e Utilizador.	Parcial	Enum <code>user_role</code> na BD e controlo de acesso na API.
RF06	Administrador: criar, editar e remover projetos.	Parcial	Ecrã Projetos: criação, detalhe, edição, eliminação, membros e tarefas.
RF07	Administrador: criar, alterar e remover perfis.	Parcial	Ecrã Admin: listagem, criação, edição, passwords, activação e eliminação.
RF08	Administrador: associar gestor a projeto.	Parcial	Definido na criação/edição do projeto.
RF09	Gestor: associar tarefas a projetos.	Parcial	Criação de tarefas no detalhe de cada projeto.
RF10	Gestor: associar utilizadores a projetos e tarefas.	Parcial	Membros no detalhe de projeto; responsáveis no detalhe de tarefa.

ID	Descrição	Estado	Notas
RF11	Gestor: visualizar tarefas concluídas e por concluir.	Parcial	Detalhe de projeto e dashboard.
RF12	Gestor: concluir projetos e avaliar performance.	Parcial	Conclusão de projetos sim; avaliação formal de performance não implementada.
RF13	Gestor: definir roles dos utilizadores no projeto.	Parcial	Roles globais apenas; sem roles por projeto.
RF14	Gestor: atribuir tempo estimado às tarefas.	Parcial	Campo <code>estimated_time</code> na criação/edição.
RF15	Disponibilizar dashboards.	Parcial	Dashboard com saudação, resumo de tarefas, secções temporais, horas da semana, alertas de tempo estimado, projetos e <i>pull-to-refresh</i> .
RF16	Utilizador: visualizar tarefas por realizar.	Parcial	Lista no dashboard e detalhe de projeto.
RF17	Utilizador: consultar histórico de tarefas concluídas.	Parcial	Histórico de registos de tempo (<code>task_time_entries</code>); sem ecrã dedicado de histórico global.
RF18	Utilizador: registar execução (data, local, taxa de conclusão, tempo).	Parcial	Data, local, tempo e observações implementados; taxa de conclusão existe no modelo mas com uso limitado na UI.

ID	Descrição	Estado	Notas
RF19	Utilizador: associar observações e fotografias.	Parcial	Registos de tempo com observação e foto; upload multi-part.
RF20	Utilizador: concluir tarefa.	Parcial	Acção no detalhe da tarefa; eliminação reservada a gestores/admins.
RF21	Visualização Kanban das tarefas.	Não implementado	Tarefas apresentadas em listas, não em colunas Kanban.
RF22	Pesquisa por projeto, utilizador ou tarefa.	Não implementado	Funcionalidade planificada mas não desenvolvida.
RF23	Exportação de estatísticas com filtros.	Parcial	Ecrã Estatísticas (via Definições): PDF por utilizador, projeto ou tarefas de projeto.

2.2 Requisitos Não Funcionais

Tabela 2: Requisitos não funcionais – estado final

ID	Descrição	Estado	Notas
RNF01	Controlo de acesso por perfil.	Parcial	Validação em serviços e routers.
RNF02	Interface simples, intuitiva e consistente.	Parcial	Design Material 3; navegação inferior coerente.
RNF03	Mensagens de erro claras.	Parcial	Toasts e mensagens localizadas EN/PT.

ID	Descrição	Estado	Notas
RNF04	Código organizado por camadas.	Parcial	Backend: entity/-repo/service/-router; frontend: UI/ViewModel/-data.
RNF05	Integridade referencial na BD.	Parcial	Chaves estrangeiras e ON DELETE CASCADE.
RNF06	Execução contentorizada com Docker.	Parcial	<code>docker-compose.yml</code> com Postgres + backend.
RNF07	Respostas da API em JSON.	Parcial	Todos os endpoints REST.
RNF08	Preparação para autenticação segura.	Parcial	Passwords com bcrypt; tokens com TTL de 7 dias.
RNF09	Evolução futura sem reestruturação drástica.	Parcial	Serviços e routers modulares.
RNF10	Estrutura adequada ao trabalho em equipa.	Parcial	Repositório Git, Trello e divisão backend/frontend.
RNF11	Internacionalização (i18n): suporte a português e inglês.	Parcial	Seleção de idioma no login e definições; strings em <code>values-pt/</code> ; bandeiras dinâmicas via API pública de países.

2.3 Requisitos adicionais identificados durante o desenvolvimento

Durante a implementação, surgiram necessidades não previstas explicitamente no relatório intermédio:

- **RFA01** – Persistência local de onboarding e fotografia de perfil offline.
- **RFA02** – Histórico detalhado de registos de tempo por tarefa (`task_time_entries`).

- **RFA03** – Atribuição de múltiplos utilizadores à mesma tarefa.
- **RFA04** – Exportação de relatórios estatísticos em PDF no backend.

3 Modelo Final da Base de Dados

A base de dados da aplicação evoluiu de forma significativa entre a entrega intermédia e a versão final do projeto. No relatório intermédio, o modelo encontrava-se ainda numa fase mais simples, com as entidades principais necessárias ao funcionamento inicial da aplicação, nomeadamente utilizadores, projetos, tarefas e a associação entre utilizadores e projetos. No entanto, à medida que a aplicação foi sendo desenvolvida e integrada com a aplicação móvel, tornou-se necessário adaptar o modelo de dados para suportar funcionalidades mais completas.

Na versão final, o modelo passou a representar melhor a realidade da aplicação, permitindo associar vários utilizadores a uma mesma tarefa, manter um histórico detalhado dos registos de tempo e guardar informação adicional relacionada com o progresso, observações e fotografias. Esta evolução tornou a estrutura da base de dados mais flexível e preparada para responder às necessidades reais do sistema.

A Figura 1 apresenta o modelo final da base de dados da aplicação COMMOV.

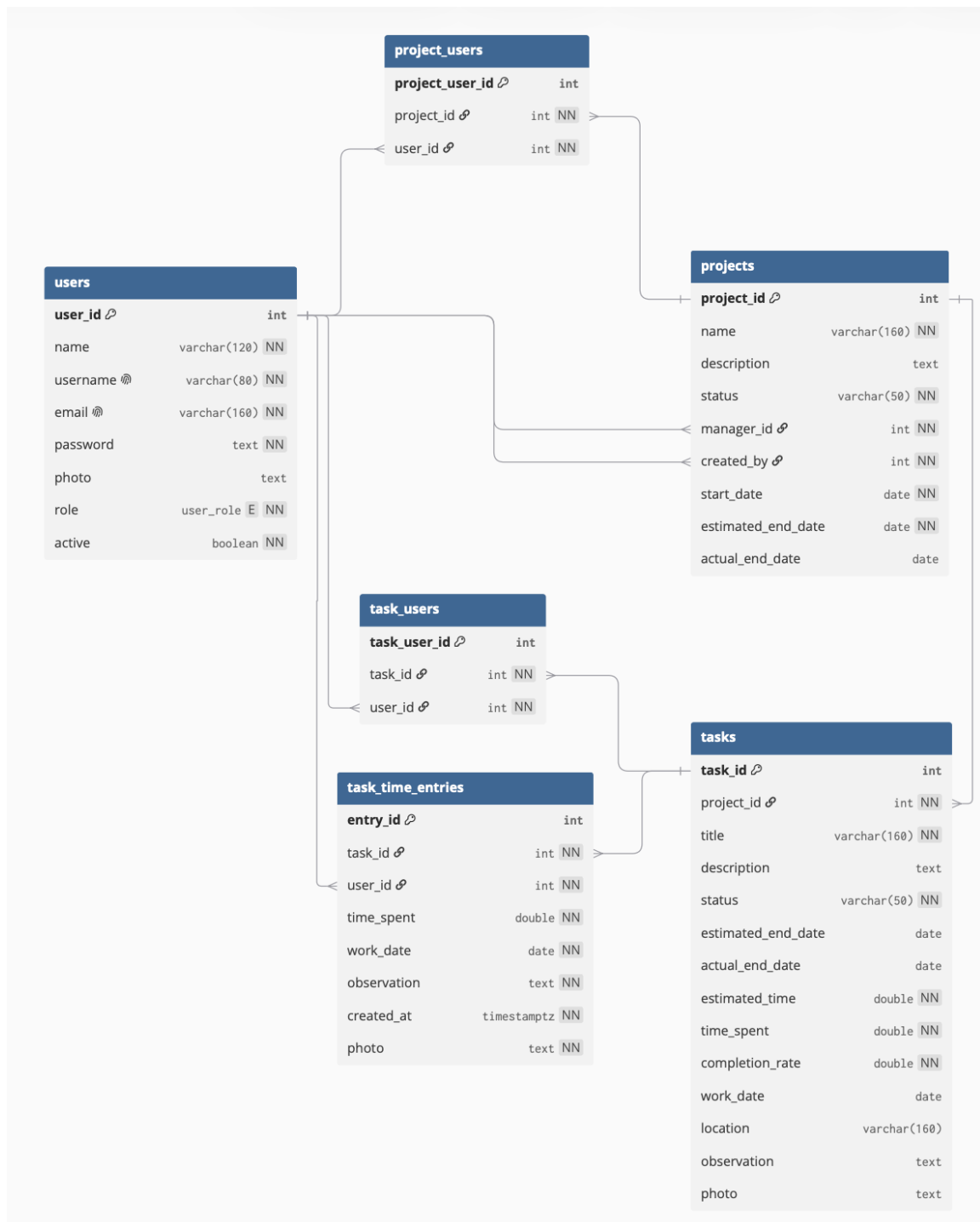


Figura 1: Modelo final da base de dados da aplicação COMMOV

3.1 Descrição das tabelas

O modelo final é composto por seis tabelas principais: **users**, **projects**, **project_users**, **tasks**, **task_users** e **task_time_entries**. Cada uma destas tabelas tem uma responsabilidade específica dentro do sistema, permitindo organizar os dados de forma clara e separada.

Tabela 3: Principais tabelas da base de dados final

Tabela	Descrição
users	Guarda os utilizadores da aplicação. Inclui dados como nome, <i>username</i> , email, <i>password</i> protegida por <i>hash</i> , fotografia, papel do utilizador e estado da conta. O campo role permite distinguir os diferentes perfis existentes no sistema, como administrador, gestor de projeto e utilizador comum.
projects	Guarda a informação dos projetos criados na aplicação. Cada projeto possui nome, descrição, estado, datas de início e fim, um utilizador responsável pela gestão do projeto e um utilizador que o criou. Desta forma, é possível saber quem criou o projeto e quem é o seu gestor.
project_users	Representa a associação entre projetos e utilizadores. Esta tabela permite que um projeto tenha vários utilizadores associados e que um utilizador possa participar em vários projetos. Funciona, por isso, como uma tabela intermédia numa relação muitos-para-muitos.
tasks	Guarda as tarefas pertencentes a cada projeto. Cada tarefa possui título, descrição, estado, data estimada de conclusão, data real de conclusão, tempo estimado, tempo gasto, percentagem de conclusão, data de trabalho, localização, observação e fotografia. Esta tabela concentra a informação principal da tarefa, mas a responsabilidade dos utilizadores e o histórico de tempo são tratados em tabelas próprias.
task_users	Representa a associação entre tarefas e utilizadores. Esta tabela foi adicionada na versão final para permitir que uma tarefa possa ter mais do que um utilizador responsável. Assim, a aplicação deixa de estar limitada a uma relação direta entre uma tarefa e um único utilizador.
task_time_entries	Guarda o histórico dos registos de tempo associados às tarefas. Cada registo indica a tarefa, o utilizador que registou o tempo, o número de horas, a data de trabalho, uma observação, uma fotografia e a data de criação do registo. Esta tabela permite acompanhar o esforço realizado ao longo do tempo, em vez de guardar apenas um valor total acumulado na tarefa.

3.2 Relações entre as tabelas

A tabela **users** está ligada a várias partes do sistema, uma vez que os utilizadores podem criar projetos, gerir projetos, participar em projetos, ser responsáveis por tarefas e registar tempo de trabalho. Na tabela **projects**, os campos **manager_id** e **created_by** fazem referência à tabela **users**, permitindo identificar o gestor do projeto e o utilizador que o criou.

A relação entre **projects** e **users** é também reforçada através da tabela **project_users**. Esta tabela permite criar uma relação muitos-para-muitos, uma vez que um projeto pode ter vários utilizadores associados e um utilizador pode estar associado a vários projetos.

As tarefas são guardadas na tabela **tasks** e estão sempre associadas a um projeto através do campo **project_id**. Desta forma, cada tarefa pertence a um projeto específico. Na versão final, a associação entre tarefas e utilizadores passou a ser feita através da tabela **task_users**, permitindo que a mesma tarefa possa ser atribuída a vários utilizadores.

Por fim, a tabela **task_time_entries** permite guardar vários registos de tempo para a mesma tarefa. Cada registo está associado a uma tarefa e a um utilizador, o que permite saber quem registou o trabalho, em que data, durante quanto tempo e com que observações ou fotografia associada.

3.3 Alterações face ao modelo intermédio

A Figura 2 representa o modelo existente na fase intermédia do projeto. Nessa fase, a estrutura da base de dados ainda era mais simples e estava sobretudo preparada para suportar as funcionalidades iniciais do *backend*.

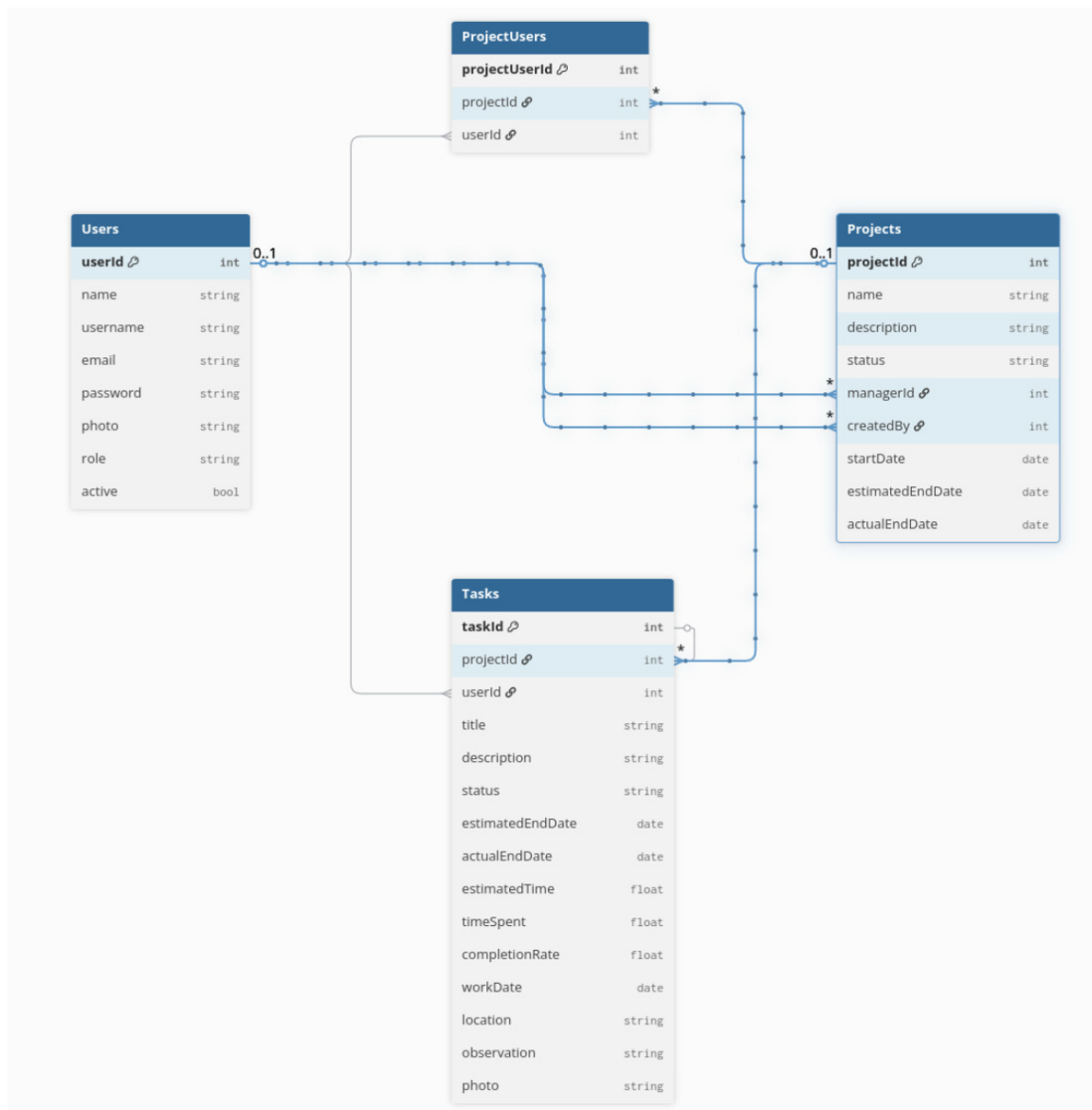


Figura 2: Modelo da base de dados na fase intermédia

Comparando o modelo intermédio com o modelo final, é possível identificar várias alterações importantes:

- A tabela **tasks** deixou de depender diretamente de um único `user_id`. No modelo final, a atribuição de utilizadores a tarefas passou a ser feita através da tabela **task_users**.
- Foi criada a tabela **task_users**, permitindo uma relação muitos-para-muitos entre tarefas e utilizadores. Esta alteração tornou possível atribuir vários responsáveis à mesma tarefa.
- Foi adicionada a tabela **task_time_entries**, responsável por guardar o histórico dos registos de tempo feitos pelos utilizadores em cada tarefa.

- A informação sobre tempo de trabalho deixou de estar limitada ao campo `time_spent` da tabela `tasks`. Com a nova tabela de registos de tempo, passou a ser possível consultar vários registos individuais associados à mesma tarefa.
- O modelo final passou a suportar observações e fotografias nos registos de tempo, tornando o acompanhamento do trabalho mais detalhado.
- A estrutura final separa melhor as responsabilidades entre as tabelas, tornando a base de dados mais organizada, flexível e preparada para futuras funcionalidades.

Estas alterações foram importantes para aproximar a base de dados das necessidades reais da aplicação. Enquanto o modelo intermédio era suficiente para uma primeira versão do *backend*, o modelo final passou a suportar melhor a lógica da aplicação móvel, especialmente no que diz respeito à atribuição de tarefas, acompanhamento do progresso e registo detalhado do trabalho realizado.

As migrações da base de dados encontram-se na pasta `backend/data/postgres/migrations/` e são aplicadas através da ferramenta **Goose**, permitindo manter o histórico de alterações da base de dados de forma organizada e reproduzível.

4 Arquitetura Final da Solução

A solução adopta uma arquitectura cliente-servidor clássica, adequada a aplicações móveis com backend centralizado.

4.1 Visão geral

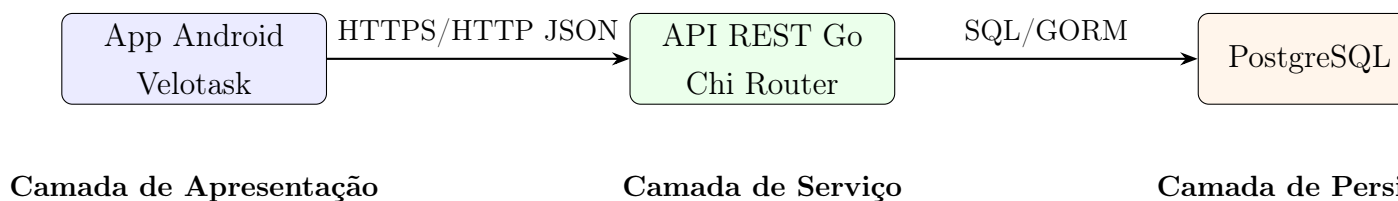


Figura 3: Arquitectura de alto nível do sistema COMMOV

4.2 Backend

O backend segue uma organização em camadas:

1. **Router** (`internal/router/`) – recebe pedidos HTTP, valida autenticação e serializa respostas JSON.

2. **Services** (`internal/services/`) – regras de negócio, autorização por perfil e validações.
3. **Repositories** (`internal/postgres/`) – acesso à base de dados via GORM.
4. **Entities** (`internal/entity/`) – modelos de domínio.

O arranque da aplicação é feito em `backend/cmd/main.go`, que inicializa a ligação PostgreSQL, os serviços, garante a existência de um utilizador administrador por defeito e expõe a API na porta 8080.

4.3 Frontend Android

A aplicação móvel combina:

- **UI** – Jetpack Compose (`ComMovScreens.kt`, `IntroScreen.kt`).
- **ViewModels** – gestão de estado e comunicação assíncrona com a API.
- **Camada de dados remota** – clientes HTTP (`AuthApi`, `ProjectsApi`, `TaskApi`, etc.).
- **Camada de dados local** – Room (onboarding), `SharedPreferences` (sessão) e armazenamento offline de fotografias.

A navegação principal utiliza uma barra inferior com quatro destinos: Início, Projetos, Administração (apenas admin) e Definições.

4.4 Autenticação e autorização

- Login via POST `/login`; resposta inclui token e dados do utilizador.
- Pedidos autenticados enviam o token no header `Authorization: Bearer <token>`.
- Tokens mantidos em memória no servidor com TTL de 7 dias; passwords armazenadas com `bcrypt`.
- Papéis determinam permissões: administradores gerem utilizadores; gestores e admins gerem projetos/tarefas; utilizadores executam tarefas atribuídas.

5 Tecnologias e Bibliotecas Utilizadas

Tabela 4: Stack tecnológico final

Componente	Tecnologia	Finalidade
Backend	Go 1.22	API REST
Router HTTP	Chi v5	Rotas e middleware
ORM	GORM + driver PostgreSQL	Persistência
Migrações	Goose	Evolução do esquema BD
Autenticação	golang.org/x/crypto/bcrypt	Hash de passwords
Base de dados	PostgreSQL 16	Armazenamento relacional
Contentorização	Docker / Docker Compose	Ambiente de execução
Frontend	Kotlin 1.9	Linguagem Android
UI	Jetpack Compose + Material 3	Interface declarativa
Imagens	Coil Compose	Carregamento de fotografias
Sessão	SharedPreferences	Token e perfil autenticado
Build	Gradle 8.9 / AGP 8.9	Compilação Android
Gestão	Git / GitHub, Trello	Versões e planeamento

6 Principais Funcionalidades Implementadas

6.1 Onboarding e autenticação

Ao abrir a aplicação pela primeira vez, o utilizador percorre quatro *intro sliders* que explicam o propósito da Velotask. O estado de visualização fica guardado localmente; nas sessões seguintes, a app avança directamente para login ou dashboard, consoante exista sessão válida.

O ecrã de login permite autenticação com email e password, validação de campos, recuperação de sessão (`/check-login`) e selecção de idioma.

6.2 Dashboard

O dashboard é o ponto central após autenticação. Apresenta:

- Saudação personalizada e avatar do utilizador;

- Resumo de tarefas por fazer e concluídas;
- Secções “Atrasadas”, “Para hoje” e “Esta semana”;
- Horas registadas na semana corrente;
- Alertas de tarefas acima do tempo estimado;
- Pré-visualização dos projetos do utilizador;
- *Pull-to-refresh* para actualizar dados.

6.3 Gestão de projetos

Utilizadores autorizados (administradores e gestores de projeto) podem criar projetos no ecrã **Projetos**, definindo gestor, membros, datas e descrição. No detalhe de cada projeto é possível:

- Consultar tarefas associadas;
- Adicionar/remover membros;
- Editar estado (activo, concluído, cancelado);
- Eliminar projeto (com confirmação);
- Criar novas tarefas.

6.4 Gestão de tarefas

Cada tarefa suporta título, descrição, data estimada, tempo estimado, localização e múltiplos responsáveis. No detalhe da tarefa, o utilizador pode:

- Registrar tempo despendido com observação e fotografia;
- Consultar histórico de registos (`task_time_entries`);
- Adicionar/remover responsáveis (gestores);
- Concluir a tarefa ou eliminá-la (eliminação reservada a gestores/admins).

6.5 Administração

O ecrã de administração, acessível apenas a administradores, permite listar utilizadores, criar contas, editar dados, alterar passwords, activar/desactivar contas (na edição) e eliminar utilizadores. Inclui métricas resumidas (total, activos, admins, gestores).

6.6 Estatísticas e exportação

Gestores e administradores acedem, através de **Definições**, a um ecrã de estatísticas com exportação PDF para:

- Relatório por utilizador;
- Relatório por projeto;
- Relatório de tarefas de um projeto.

Os PDFs são gerados no backend e guardados/abertos no dispositivo Android.

6.7 Definições

O ecrã de definições permite alterar fotografia de perfil, escolher idioma (português/inglês), consultar dados da conta, aceder a estatísticas (se autorizado) e terminar sessão.

7 Funcionalidades Extra Desenvolvidas

Para além do âmbito inicial, o grupo implementou funcionalidades complementares:

1. **Fotografia offline antes do login** – permite seleccionar foto de perfil antes de autenticação; a imagem fica pendente e sincroniza após login.
2. **Sincronização de fotografia de perfil** – `ProfilePhotoSyncManager` envia fotos pendentes e notifica o utilizador.
3. **Persistência de onboarding com Room** – evita repetir os *intro sliders* desnecessariamente.
4. **Histórico de registos de tempo** – cada entrada de tempo é individualizada, com foto e observação.

5. **Atribuição múltipla de tarefas** – vários utilizadores podem estar associados à mesma tarefa.
6. **Esqueletos de carregamento (*skeleton screens*)** – melhoram a percepção de performance durante fetch de dados.
7. **Restauro de sessão** – verificação automática de token ao arrancar a aplicação.

8 Justificação das Alterações face à Entrega Intermédia

A evolução do projecto reflecte decisões técnicas tomadas à medida que o desenvolvimento avançou:

- **De backend mínimo para API completa** – na fase intermédia existiam apenas `/health` e `POST /users`. A versão final inclui autenticação, CRUD de utilizadores, projetos, tarefas, upload de fotografias e exportação PDF, necessários para a app móvel funcionar de forma autónoma.
- **Implementação do frontend Android** – previsto no relatório intermédio mas ainda inexistente; passou a ser o cliente principal, com Compose em vez de XML puro, alinhando-se com as boas práticas actuais de desenvolvimento Android.
- **Modelo de dados alargado** – a tabela `task_users` surgiu da necessidade real de equipas partilharem tarefas. `task_time_entries` permitiu histórico auditável de execução, mais adequado do que sobrescrever campos na tabela `tasks`.
- **Simplificação de estados de tarefa** – estados intermédios (`pending`, `in_progress`) foram consolidados em `todo/completed` para reduzir complexidade na UI móvel, mantendo estados de projecto mais ricos (`active`, `completed`, `on_hold`, `cancelled`).
- **Autenticação por token em memória** – solução pragmática para o âmbito académico; cumpre RNF08 sem introduzir JWT/OAuth.
- **Registo de contas pelo administrador** – em vez de registo público, optámos por criação controlada de contas, reforçando o controlo de perfis num contexto empresarial simulado.
- **Nome comercial Velotask** – mantém a identidade visual dos mockups, enquanto o repositório e backend conservam a designação académica COMMOV.
- **Funcionalidades adiadas** – Kanban (RF21) e pesquisa global (RF22) foram postergadas para priorizar um núcleo funcional sólido dentro do prazo disponível.

9 Capturas de Ecrã da Aplicação

Esta secção apresenta uma comparação visual entre os mockups originais desenhados no Figma e a implementação final da aplicação Velotask. Em cada figura, o ecrã à **esquerda** corresponde ao protótipo de interface; o ecrã à **direita** mostra a captura real obtida no emulador Android.

Os ficheiros de imagem devem ser colocados na pasta `figures/`, com o prefixo `figma_` para os prints do Figma e `ecra_` para as capturas da app (ver `figures/README.md`).

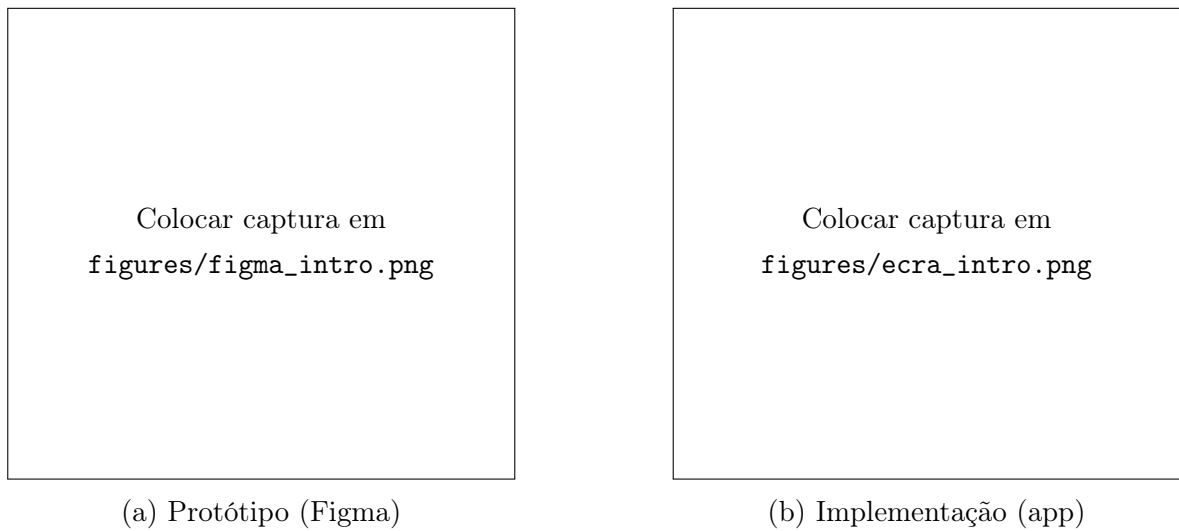


Figura 4: Ecrã de onboarding / intro sliders

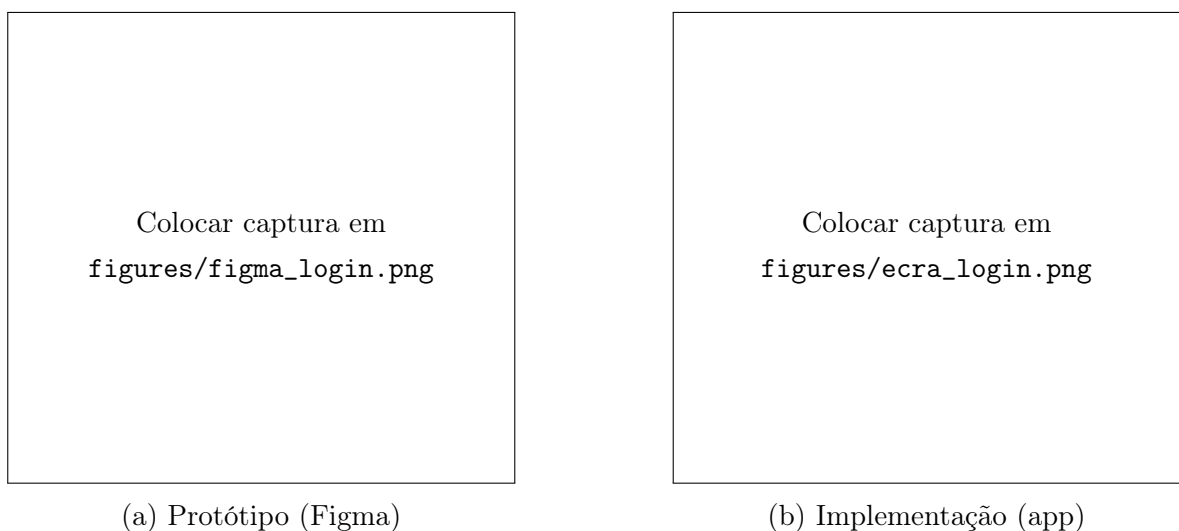
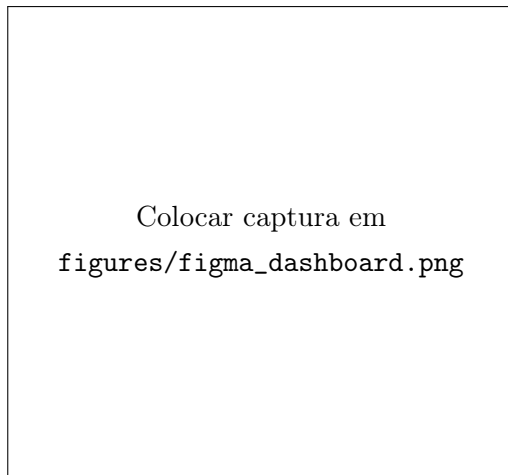
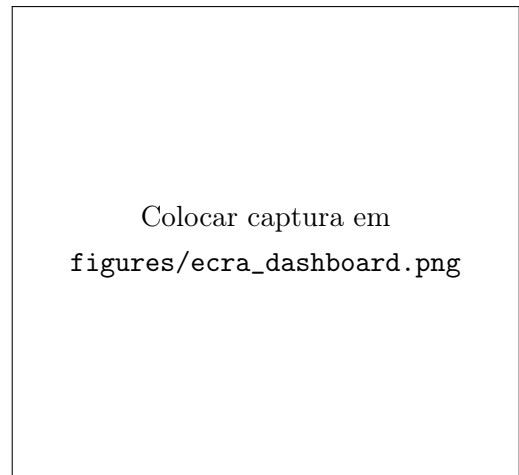


Figura 5: Ecrã de início de sessão

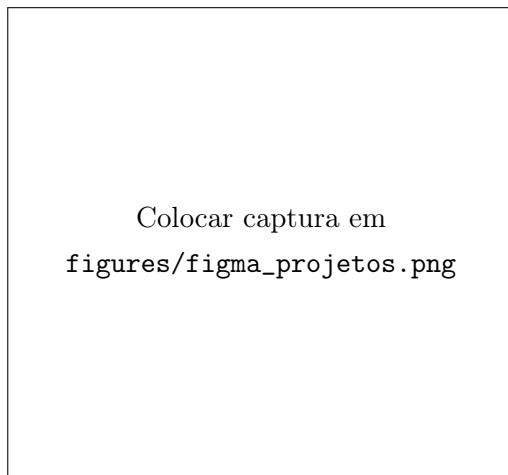


(a) Protótipo (Figma)

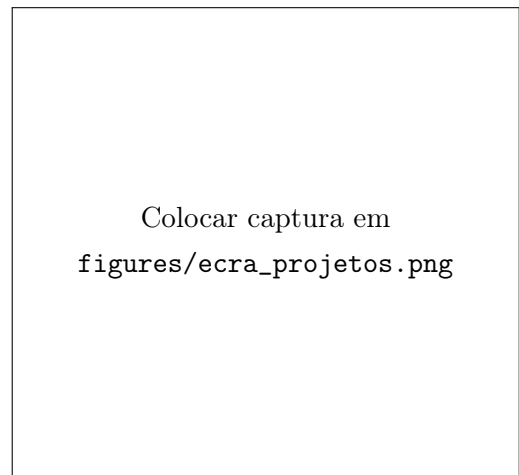


(b) Implementação (app)

Figura 6: Dashboard principal

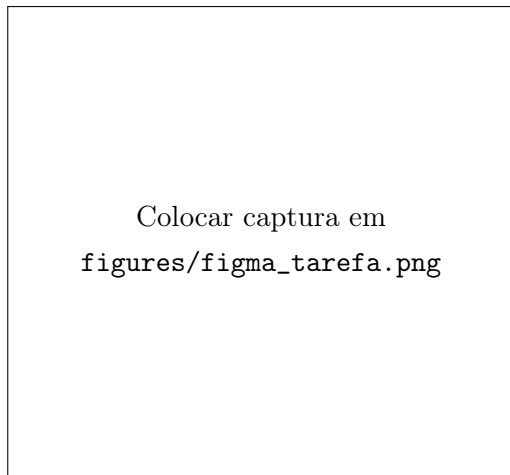


(a) Protótipo (Figma)

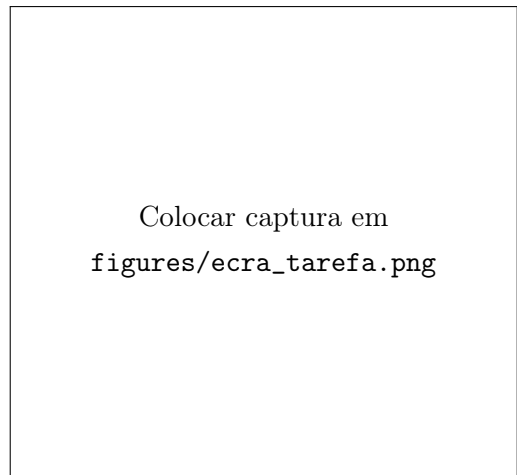


(b) Implementação (app)

Figura 7: Lista de projetos

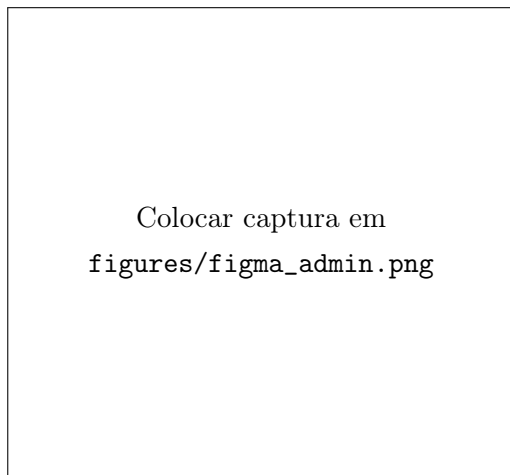


(a) Protótipo (Figma)

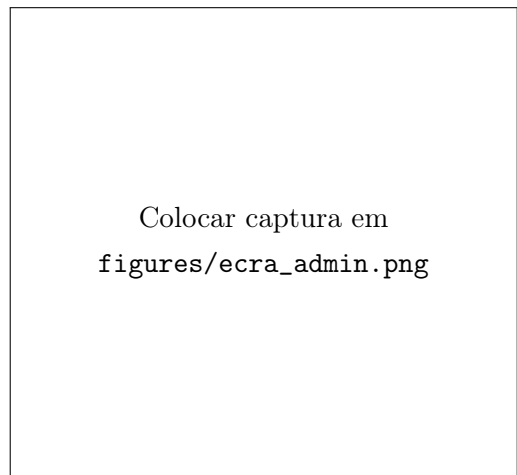


(b) Implementação (app)

Figura 8: Detalhe de tarefa com registo de tempo



(a) Protótipo (Figma)



(b) Implementação (app)

Figura 9: Administração de utilizadores

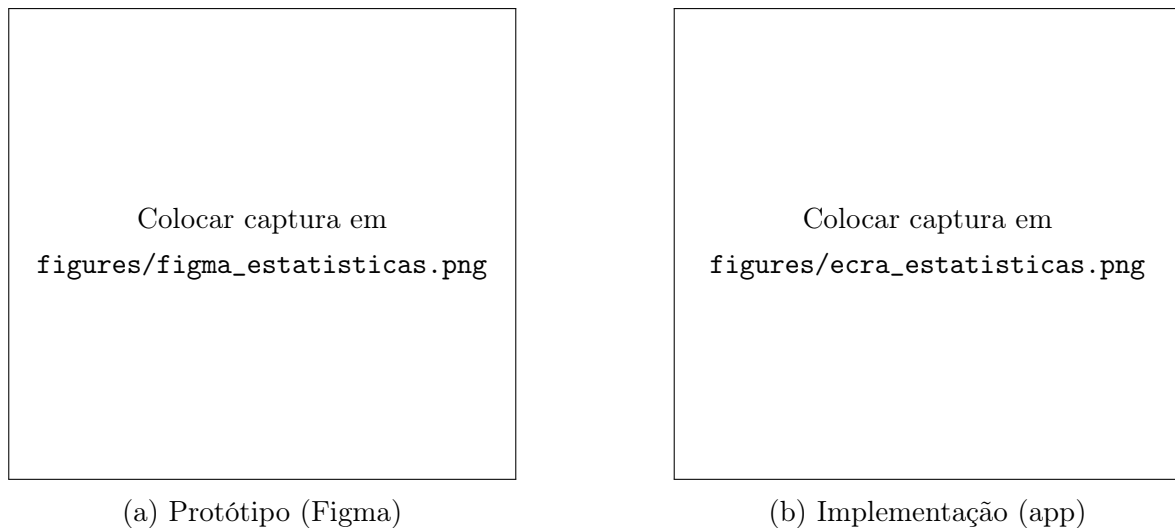


Figura 10: Exportação de estatísticas em PDF

10 Ligações de Apoio

10.1 Mockups e protótipos (Figma)

Os mockups originais do projecto encontram-se disponíveis em:

<https://www.figma.com/design/o6ar6AI25khe4mwHBM6xve/Untitled?node-id=0-1&p=f>

10.2 Repositório Git

Código-fonte, histórico de commits e documentação:

<https://github.com/rafaeldiogomonteiro/COMMOV>

10.3 Plataforma de gestão de projecto (Trello)

Planeamento e acompanhamento de tarefas da equipa:

<https://trello.com/b/SEU-QUADRO-TRELLO-AQUI>

11 Testes e Validações Efectuadas

11.1 Testes automatizados no backend

Foram implementados testes unitários em Go, executáveis com:

```
cd backend  
go test ./...
```

Os testes cobrem, entre outros:

- Expiração e revogação de tokens (`auth_service_test.go`);
- Normalização de estados de projecto e validação de datas (`project_service_test.go`);
- Estados de entidades (`status_test.go`, `user_test.go`);
- Endpoint de saúde (`health_test.go`).

11.2 Testes no frontend

Existem testes unitários básicos em `StatusTest.kt` para normalização de estados de tarefa. Testes instrumentados (Espresso) encontram-se configurados mas com cobertura limitada.

11.3 Validação manual

A validação funcional foi efectuada de forma manual através de:

- Emulador Android Studio ligado ao backend Docker local;
- Cenários por perfil: admin, gestor de projecto e utilizador comum;
- Verificação de fluxos críticos: login, CRUD, registo de tempo, conclusão de tarefa, exportação PDF e troca de idioma.

11.4 Resultados gerais

Os fluxos principais funcionam de forma consistente. Foram identificadas limitações conhecidas: ausência da vista Kanban e registo autónomo de utilizadores. Estas funcionalidades ficam registadas como evolução futura.

12 Instruções de Instalação, Execução e Utilização

12.1 Pré-requisitos

- Docker e Docker Compose;
- Go 1.22+ (opcional, para desenvolvimento local);
- Android Studio (Koala ou superior recomendado);
- Emulador Android API 24+ ou dispositivo físico na mesma rede.

12.2 Arranque do backend

```
cd backend
docker compose up -d --build
```

Serviços disponíveis:

- API: <http://localhost:8080>
- Health check: GET /health
- PostgreSQL: localhost:5432 (user/pass: postgres/postgres)

Por defeito, é criado um administrador configurável via `.env`:

```
DEFAULT_USER=admin@commov.local
DEFAULT_USER_PASS=admin123
```

Migrações manuais (se necessário):

```
cd backend
make goose-up
```

12.3 Configuração e execução da app Android

1. Criar/editar `frontend/.env`:

```
API_BASE_URL=http://10.0.2.2:8080
```

Nota: 10.0.2.2 é o endereço do localhost visto a partir do emulador Android.

2. Abrir a pasta `frontend/` no Android Studio.
3. Sincronizar Gradle e executar a app num emulador ou dispositivo.

12.4 Utilização básica

1. Na primeira execução, percorrer os *intro sliders* ou saltar.
2. Iniciar sessão com credenciais válidas (ex.: admin por defeito).
3. Explorar o dashboard e aceder a “Projetos”.
4. Administradores: criar utilizadores em “Admin”.
5. Gestores/admins: criar projectos, adicionar membros e tarefas.
6. Utilizadores: abrir tarefas atribuídas, registar tempo e concluir trabalho.
7. Gestores/admins: exportar PDFs em “Estatísticas” (acessível via definições ou menu).
8. Alterar idioma em “Definições” ou no ecrã de login.
9. Terminar sessão nas definições.

13 Conclusão

O projeto COMMOV evoluiu significativamente desde a fase intermédia, passando de uma base backend inicial para uma solução móvel integrada e funcional. Na fase anterior, o sistema já apresentava uma fundação técnica consistente, com uma arquitetura organizada por camadas, ligação à base de dados PostgreSQL, migrações iniciais, repositórios de dados e endpoints básicos para validação da API. Essa estrutura permitiu dar continuidade ao desenvolvimento de forma mais organizada e sustentada. Na versão final, essa base foi consolidada e expandida através da implementação da aplicação Android Velotask, que permite suportar o ciclo de vida de projetos e tarefas em contexto móvel. O sistema passou a integrar diferentes perfis de utilizador, autenticação, gestão de projetos, gestão de tarefas, associação de utilizadores a projetos e tarefas, bem como funcionalidades adicionais como o registo e histórico de tempo e a exportação de relatórios em PDF. O modelo de dados também evoluiu relativamente à fase intermédia, passando a representar de forma mais completa as relações entre utilizadores, projetos, tarefas e registos de tempo. Esta evolução permitiu responder de forma mais adequada aos requisitos funcionais definidos, garantindo uma persistência relacional mais robusta e coerente com as necessidades da aplicação. Apesar de a maior parte dos objetivos inicialmente definidos ter sido concretizada, continuam a existir oportunidades de melhoria.

Em suma, o projeto COMMOV demonstra uma evolução clara entre a fase intermédia e a versão final. O sistema apresenta coerência técnica, uma arquitetura adequada, boa integração entre backend e aplicação móvel e uma base sólida para futuras melhorias. A

solução desenvolvida cumpre o seu objetivo principal: disponibilizar uma aplicação móvel funcional para apoio à gestão de projetos, tarefas e tempo de trabalho. **Repositório:**

<https://github.com/rafaeldiogomonteiro/COMMOV>